

GUÍA DE DEPLOY

Servidor de Clase 2026

React + Node.js + Prisma + SQLite

Manual técnico de referencia para el empaquetado, transferencia y despliegue automatizado de aplicaciones full-stack. Incluye configuraciones de entorno para entornos de producción controlados mediante procesos en segundo plano (PM2).

Autor: Ignacio Joaquín
Materia: Aplicaciones Específicas de Redes
Fecha: 17 de mayo de 2026

GUÍA DE DEPLOY

Cómo subir tu aplicación al servidor

React + Node.js + Prisma + SQLite

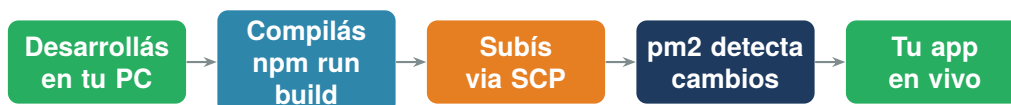
Datos del servidor

Dato	Valor
Dirección IP	200.3.127.46
Puerto web	8002
Puerto SSH	22
Tu sitio web	http://200.3.127.46:8002/~USUARIO/
Tu API	http://200.3.127.46:8002/~USUARIO/api/

Usuarios y contraseñas

Usuario	Contraseña	Tu URL
uno	*****	http://200.3.127.46:8002/~uno/
dos	*****	http://200.3.127.46:8002/~dos/
tres	*****	http://200.3.127.46:8002/~tres/
cuatro	*****	http://200.3.127.46:8002/~cuatro/
cinco	*****	http://200.3.127.46:8002/~cinco/
seis	*****	http://200.3.127.46:8002/~seis/
siete	*****	http://200.3.127.46:8002/~siete/
ocho	*****	http://200.3.127.46:8002/~ocho/
nueve	*****	http://200.3.127.46:8002/~nueve/
diez	*****	http://200.3.127.46:8002/~diez/

Flujo de deploy



[!] IMPORTANTE: El servidor ya tiene node_modules instalado. Solo subís dist/ del backend y los archivos de build/ o dist/ del frontend. **Nunca subas node_modules/.**

PASO 1 — Configurar el Frontend (React)

Antes de compilar, tenés que indicarle a React cuál es la URL base de tu app. Sin esto los assets JS y CSS no van a cargar desde el servidor.

Si usás Vite — modificá vite.config.ts

[i] Reemplazá dos con **tu usuario** en todos los ejemplos.

```
// Reemplaza "dos" con TU usuario
export default defineConfig({
  base: '~/dos/',
  // ... resto de tu configuracion
})
```

Si usás Create React App — modificá package.json

```
{
  "homepage": "~/dos/",
  "name": "mi-app",
  // ... resto de tu package.json
}
```

URL de la API — detección automática

En vez de hardcodear localhost:3001, usá este snippet para que funcione tanto local como en el servidor:

```
// Detecta si estas en el servidor o en local
export const API_BASE = window.location.pathname.startsWith('/')
  ? `/${window.location.pathname.split('/')[1]}/api`
  : 'http://localhost:3001';

// Uso:
// fetch(`${API_BASE}/tarefas`)
// fetch(`${API_BASE}/productos`)
```

Compilar el frontend

```
# En la carpeta raiz de tu proyecto React
npm run build

# Vite genera: dist/
# CRA genera: build/
```

PASO 2 — Subir el Frontend al Servidor

[!] Subí el **contenido** de dist/ o build/, no la carpeta entera. Fijate en el /* al final del comando.

```
# Si usas Vite (carpeta dist/)
scp -r dist/* dos@200.3.127.46:~/public_html/

# Si usas Create React App (carpeta build/)
scp -r build/* dos@200.3.127.46:~/public_html/

# Te va a pedir tu contraseña (ej: Cohete2026)
```

✓ Tu sitio queda disponible en: <http://200.3.127.46:8002/~dos/>

× Si ves página en blanco o error 403, revisá que hayas subido el *contenido* (archivos sueltos), no la carpeta dist/ completa.

PASO 3 — Configurar el Backend (Node.js + Prisma)

La estructura de tu proyecto backend tiene que ser así:

```
servicios/
|-- src/
|   |-- index.ts           <- tu código fuente TypeScript
|-- dist/                  <- generado por npm run build (esto subís)
|   |-- index.js
|-- prisma/
|   |-- schema.prisma      <- tu esquema de base de datos
|   |-- migrations/       <- generado por prisma migrate dev
|-- package.json
|-- tsconfig.json
```

tsconfig.json — usar CommonJS (IMPORTANTE)

× El servidor usa CommonJS. Si tu tsconfig tiene "module": "ESNext" va a fallar. Usá esta configuración exacta.

```
tsconfig.json
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
}
```

```
"include": ["src/**/*"],
"exclude": ["node_modules", "dist"]
}
```

Importar Prisma correctamente en CommonJS

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import pkg from '@prisma/client'; // así se importa en CommonJS
const { PrismaClient } = pkg;

dotenv.config();
const app = express();
const prisma = new PrismaClient();
const PORT = Number(process.env.PORT) || 3001;

app.use(cors());
app.use(express.json());

// tus rutas aca...

app.listen(PORT, '0.0.0.0', () =>
  console.log(`API corriendo en puerto ${PORT}`)
);
```

Variables de entorno (.env) — solo en tu computadora

[i] El servidor ya tiene el .env configurado con tu puerto y base de datos. **No subas tu .env local.**

```
PORT=3001
DATABASE_URL="file:./dev.db"
```

PASO 4 — Prisma y Base de Datos

El servidor usa **SQLite**: una base de datos en un solo archivo. No necesitas instalar nada ni configurar credenciales.

Definir tu esquema

```
prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

// Ejemplo de modelo:
```

```
model Tarea {  
  id      Int      @id @default(autoincrement())  
  titulo String  
  hecho   Boolean @default(false)  
}
```

Crear migraciones — en tu computadora

[i] Hacé esto en tu computadora. El servidor aplica las migraciones automáticamente cuando subís los archivos.

```
# Primera vez o cuando cambias el schema:  
npx prisma migrate dev --name nombre_migracion  
  
# Ejemplos:  
npx prisma migrate dev --name init  
npx prisma migrate dev --name agrego_columna_precio  
npx prisma migrate dev --name nueva_tabla_usuarios
```

Subir las migraciones al servidor

```
# Subir toda la carpeta prisma (schema + migrations)  
scp -r prisma/* dos@200.3.127.46:~/servicios/prisma/  
  
# El servidor detecta las nuevas migraciones,  
# las aplica automáticamente y reinicia tu backend.
```

[!] Si cambiás el schema sin crear una migración, los cambios **no** se van a aplicar en el servidor. Siempre usá `prisma migrate dev` primero.

PASO 5 — Compilar y Subir el Backend

Compilar

```
npm run build  
  
# Genera dist/ con tu index.js compilado
```

Subir el dist compilado

```
scp -r dist/* dos@200.3.127.46:~/servicios/dist/  
  
# pm2 detecta el cambio y reinicia tu backend en ~2 segundos
```

✓ Tu API queda disponible en: `http://200.3.127.46:8002/~dos/api/`

PASO 6 — Conectarse via SSH

SSH te da una terminal en el servidor. Lo necesitás para instalar nuevos paquetes npm o revisar errores.

Conectarse

```
ssh dos@200.3.127.46
# Contraseña: Cohete2026
```

Instalar un nuevo paquete npm

```
cd ~/servicios
npm install nombre-del-paquete

# Ejemplos:
npm install bcrypt
npm install jsonwebtoken
npm install multer
npm install axios
```

[i] Después de instalar un paquete, volvé a compilar (`npm run build`) y subí el `dist/` actualizado.

Ver logs y estado del backend

```
# Ver logs en tiempo real
pm2 logs servicios

# Ver los ultimos 50 lineas
pm2 logs servicios --lines 50

# Ver estado
pm2 list

# Reiniciar manualmente
pm2 restart servicios
```

RESUMEN RÁPIDO — Cheat Sheet

Qué querés hacer	Comando
Compilar frontend (Vite)	<code>npm run build</code>
Subir frontend (Vite)	<code>scp -r dist/* dos@200.3.127.46:~/public_html/</code>
Compilar backend	<code>npm run build</code> (en carpeta servicios/)
Subir backend	<code>scp -r dist/* dos@200.3.127.46:~/servicios/dist/</code>
Subir migraciones Prisma	<code>scp -r prisma/* dos@200.3.127.46:~/servicios/prisma/</code>
Crear migración	<code>npx prisma migrate dev --name nombre</code>
Conectarse por SSH	<code>ssh dos@200.3.127.46</code>
Instalar paquete npm	<code>npm install paquete</code> (por SSH en ~/servicios)
Ver logs del backend	<code>pm2 logs servicios</code> (por SSH)
Reiniciar backend	<code>pm2 restart servicios</code> (por SSH)

Problemas comunes

Problema	Solución
Página en blanco	Revisá que base u homepage esté configurado (Paso 1)
Error 404 en assets JS/CSS	El base URL no está configurado — revisá el Paso 1
Error 502 Bad Gateway	Tu backend no corre — conectate por SSH y revisá <code>pm2 logs servicios</code>
Cannot use import statement	Tu tsconfig tiene <code>module: ESNext</code> — cambialo a <code>commonjs</code>
Module not found: express	Conectate por SSH, <code>cd ~/servicios</code> y corré <code>npm install</code>
Migraciones no se aplican	Subí la carpeta <code>prisma/</code> completa con <code>scp -r prisma/*</code>
Contraseña incorrecta en SSH	Verificá la tabla de usuarios en esta guía

[i] Tenés algún problema que no está acá? Conectate por SSH y revisá los logs con:
`pm2 logs servicios`